



北京航空航天大学学报
Journal of Beijing University of Aeronautics and Astronautics
ISSN 1001-5965, CN 11-2625/V

《北京航空航天大学学报》网络首发论文

题目: 面向边缘端 CNN 加速的矢量处理器设计与实现
作者: 孙楷, 陈亦欧
DOI: 10.13700/j.bh.1001-5965.2025.0500
收稿日期: 2025-07-18
网络首发日期: 2026-03-23
引用格式: 孙楷, 陈亦欧. 面向边缘端 CNN 加速的矢量处理器设计与实现[J/OL]. 北京航空航天大学学报. <https://doi.org/10.13700/j.bh.1001-5965.2025.0500>



网络首发: 在编辑部工作流程中, 稿件从录用到出版要经历录用定稿、排版定稿、整期汇编定稿等阶段。录用定稿指内容已经确定, 且通过同行评议、主编终审同意刊用的稿件。排版定稿指录用定稿按照期刊特定版式(包括网络呈现版式)排版后的稿件, 可暂不确定出版年、卷、期和页码。整期汇编定稿指出版年、卷、期、页码均已确定的印刷或数字出版的整期汇编稿件。录用定稿网络首发稿件内容必须符合《出版管理条例》和《期刊出版管理规定》的有关规定; 学术研究成果具有创新性、科学性和先进性, 符合编辑部对刊文的录用要求, 不存在学术不端行为及其他侵权行为; 稿件内容应基本符合国家有关书刊编辑、出版的技术标准, 正确使用和统一规范语言文字、符号、数字、外文字母、法定计量单位及地图标注等。为确保录用定稿网络首发的严肃性, 录用定稿一经发布, 不得修改论文题目、作者、机构名称和学术内容, 只可基于编辑规范进行少量文字的修改。

出版确认: 纸质期刊编辑部通过与《中国学术期刊(光盘版)》电子杂志社有限公司签约, 在《中国学术期刊(网络版)》出版传播平台上创办与纸质期刊内容一致的网络版, 以单篇或整期出版形式, 在印刷出版之前刊发论文的录用定稿、排版定稿、整期汇编定稿。因为《中国学术期刊(网络版)》是国家新闻出版广电总局批准的网络连续型出版物(ISSN 2096-4188, CN 11-6037/Z), 所以签约期刊的网络版上网络首发论文视为正式出版。

面向边缘端 CNN 加速的矢量处理器设计与实现

孙楷, 陈亦欧

(电子科技大学 通信抗干扰全国重点实验室, 成都 611731)

*通信作者 陈亦欧, E-mail: chenyou@uestc.edu.cn

摘要 人工智能物联网 (AIoT) 的兴起正驱动神经网络的部署从云端向边缘端迁移, 以满足应用对低延迟和数据隐私日益增长的需求。针对此场景下, 边缘设备部署卷积神经网络面临的性能、资源与灵活性的挑战, 设计了一种应用特定指令集处理器 (Application-Specific Instruction-Set Processor, ASIP)。该设计通过融合数据级与指令级并行, 并将卷积计算映射为细粒度的专用指令, 实现了高吞吐量与对不同网络结构的灵活适应性。进一步提出一种基于数据层链接重构并行计算方法, 基于处理器矢量架构, 高效处理任意尺寸的常规、深度及逐点卷积, 同时简化访存并免除零填充操作。此外, 设计中集成了采用乒乓缓冲机制的 DMA 控制器, 以重叠数据传输和计算任务的时间, 提升了片上资源利用率且降低整体推理时延。该处理器基于 TSMC 28nm 工艺实现, 核心面积 1.229 mm²。在 1.0 GHz 主频下, 峰值性能达 130 GOPs, 面积与门电路效率分别为 105.78 GOPs/mm²和 53.32 GOPs/MGE, 运行 MobileNetV1 时能效为 332 GOPs/W。结果表明, 本文所提出的 ASIP 架构为边缘 AI 应用提供了一个高性能、高能效的灵活硬件方案。

关键词 卷积神经网络; 应用特定指令集处理器; 硬件加速; 高能量效率; 高面积效率

中图分类号 TP387; TN492

文献标志码 A

DOI: 10.13700/j.bh.1001-5965.2025.0500

Design and implementation of vector processor for edge CNN acceleration

SUN Kai, CHEN Yiou

(National Key Laboratory of Wireless Communications, University of Electronic Science and Technology of China, Chengdu 611731, China)

*E-mail: chenyou@uestc.edu.cn

Abstract The rise of the Artificial Intelligence Internet of Things (AIoT) is driving the migration of neural network deployments from the cloud to the edge to meet the growing demand for low latency and data privacy. In order to solve the challenges of performance, resources and flexibility faced by edge devices in deploying convolutional neural networks in this scenario, an Application-Specific Instruction-Set Processor (ASIP) was designed. By fusing data-level and instruction-level parallelism, and mapping convolutional computation to fine-grained dedicated instructions, the design achieves high throughput and flexible adaptability to different network structures. Furthermore, a parallel computing method based on data layer link reconstruction is proposed, which is based on the processor vector architecture, which can efficiently process the conventional, deep and point-by-point convolution of any size, and at the same time simplify the memory access and avoid the zero-fill operation. In addition, a DMA controller with a ping-pong buffering mechanism is integrated into the design to overlap the time of data transmission and computing tasks, improve on-chip resource utilization and reduce the overall inference delay. The processor is based on TSMC's 28nm process and has a core area of 1.229 mm². At 1.0 GHz, the peak performance is 130 GOPs, the area and gate efficiency are 105.78 GOPs/mm² and 53.32 GOPs/MGE, respectively, and the efficiency is 332 GOPs/W when running MobileNetV1. The results show that the ASIP architecture proposed in this paper provides a high-performance, energy-efficient and flexible hardware solution for edge AI applications.

Key words convolutional neural network; application-specific instruction set processors; hardware acceleration; high energy efficiency; high area efficiency

近年来, 随着人工智能 (AI) 技术的快速发展, 卷积神经网络 (CNN) 作为其核心组成部分, 在

计算机视觉、自动驾驶、医疗影像分析等领域展现出卓越性能,已成为推动相关技术发展的重要力量。当前,大多数高性能 CNN 模型的部署仍依赖于具备强大计算能力的服务器端。然而,由于物联网(IoT)设备的激增以及对数据处理实时性、隐私安全性的需求,促使越来越多的 AI 应用向边缘端设备迁移。在边缘端部署神经网络能够有效降低数据传输延迟、保护用户隐私并提高系统的响应速度^[1-3]。

但将 CNN 模型部署在资源受限的边缘端设备上面临着算力-资源-灵活性的三重挑战。CNN 模型需要进行大量的乘加密集型的卷积运算,然而,边缘端设备由于功耗、散热和硬件资源的严格限制,其处理器的计算性能远不及服务器端;其次,现代 CNN 模型通常有数百万甚至数十亿的参数,并产生大量中间计算数据,这对设备的存储容量以及内存带宽提出了极大的要求^[4];并且随着深度学习技术的迅速发展,新的网络模型结构层出不穷,边缘端硬件平台需要具备足够的灵活性和可编程性以适应各种新兴的 CNN 模型^[5]。

因此,研究如何在严格资源约束的条件下,实现 CNN 在边缘终端设备上高效且低功耗的推理,是当前 AIoT 领域亟需解决的问题。

在 CNN 加速器研究领域,先前的工作主要集中于专用集成电路(ASIC)^[6-16]和现场可编程门阵列(FPGA)^[17-21]。尽管 ASIC 能通过针对特定的任务深度优化实现极高的能效,但这是以大量的硬件资源开销和极低的灵活性为代价的。基于 FPGA 的加速器虽然提供了更高的吞吐量和更大的灵活性,但在能效方面通常不及 ASIC,且开发周期长,开发成本高。

Eyeriss^[6]是一款著名的 CNN 推理 ASIC,采用 Row stationary 数据流计算规则,优化数据访问模式,显著降低了高成本 DRAM 和 GLB 的访问开销,实现高能效,但是其固定架构和广播式片上网络无法提供最佳的灵活性和效率,不适合边缘计算。Origami^[7]是一款高性能、面效能俱佳的 CNN 加速器,通过并行处理多个输入和输出通道,显著降低了外部存储带宽需求,达到 TOP 级的算力,但其图像边缘效应会导致对小尺寸图像处理效率下降,而边缘设备经常需要处理来自传感器的小尺寸图像或视频帧,因此在边缘设备应用时,实际性能会下降。Eyeriss v2^[8]是专为紧凑型 and 稀疏深度神经网络设计的灵活加速器架构,其分层网格片上网络的架构能适应 DNN 中广泛变化的滤波器形状和尺寸,以及稀疏 DNN 中变化的数据稀疏性,但是为了支持稀疏处理,PE 阵列的设计变得更加复杂,导致面积显著增加。Ko-Chi Kuo^[9]等人通过 PE 设计和卷积核尺寸裁剪,实现了不同核尺寸下 100% 的 PE 硬件利用率,但其 MACs 数量较少,峰值性能有限,面对稍复杂的边缘计算任务,其峰值性能可能不足以满足需求。

为解决以上问题,本文设计了一款应用特定指令集处理器(ASIP),平衡了计算效率和灵活性。通过软硬件协同优化为边缘端 CNN 部署提供了兼具通用性和高效性的解决方案,尤其适合需要支持多模型、低功耗的移动或 AIoT 场景。总的来说,本文的主要贡献有以下几个方面:

1) 高效的并行计算架构:基于超长指令字(Very Long Instruction Word,VLIW)加单指令多数据(Single Instruction Multiple Data,SIMD)加 7 级流水线的架构,设计了一款能实现高效并行计算的 ASIP 处理器,并且针对 CNN 中卷积运算和全连接需要多层循环套用的计算特点,设计零开销循环模块,显著减少分支指令的执行,避免了流水线停顿造成的性能损失和控制冒险,减少神经网络推理时间。

2) 精简灵活的指令集系统:ASIP 采用基于专用指令集的方式加速 CNN 中的关键运算。它不依赖固定的二维矩阵处理阵列,而是将核心算子映射为可通过汇编原语访问的细粒度专用指令,使得 ASIP 通过设计专用指令集不仅能高效执行包括标准卷积、深度可分离卷积以及点卷积在内的多种卷积计算,同时也能高效执行池化操作和全连接层的矩阵向量乘法。结合算法级的优化,可进一步适应不同参数配置的计算需求。

3) 灵活高效的并行卷积算法:本文提出一种“数据层链接并行重构计算”(Data-Layer Linking Reconstruction Parallel,DLLRP)方法,旨在优化 ASIP 架构下 CNN 的计算效率。该方法解决了卷积核滑动操作引起的非连续、高开销访存问题。通过在计算过程中链接重构数据,DLLRP 显著降低了访

存次数,提升了数据的复用性,并且基于此算法设计的专用指令集能够高效映射到 ASIP 并行架构上,支持所有网络层的高并行计算,大幅减少了神经网络的推理时延。

4) 高效的片上传输与管理机制:为解决 CNN 模型参数量庞大与片上存储资源有限之间的矛盾,设计了一个支持标准 AXI 接口的 DMA 模块,负责与 ASIP 进行控制信息交互,与外部存储之间进行数据传输。通过乒乓 Buffer 的传输机制, DMA 可以在 ASIP 处理当前数据块的同时,并行地预取下一组数据块到另一缓冲区。有效减少了计算核心对片上存储容量的瞬时需求,并通过隐藏数据的加载延迟,显著提高了 ASIP 的计算效率。

本文第二节介绍了 CNN 核心算法及关键算子分析,第三节详细阐述了本文所设计 ASIP 的整体架构以及各功能单元设计,第四节基于所提出的 ASIP 架构,介绍了 DLLRP 算法的具体实现细节,第五节为实验部分,将 MobileNetV1 神经网络部署到本文 ASIP 上进行了硬件实现,进行性能分析,并与现有的经典加速器和商用的加速器进行对比,第六节对全文进行了总结。

1 CNN 核心算法分析

为实现 CNN 在 ASIP 上的高效部署,特别是针对计算密集层的加速,同时最大化硬件资源复用率,本节对 CNN 的核心运算如常规卷积、池化、全连接以及轻量化神经网络中关键的深度可分离卷积进行了详细的算法分析。揭示了这些运算的共性计算特征,为后续设计专用指令集提供支撑,从而在不显著增加硬件资源的前提下,通过专用指令实现对上述各类运算高效并行加速。

1.1 常规卷积

常规卷积是 CNN 中最基础且计算最密集的操作,如图 1 所示。其过程可描述为:一组维度为 $K_C \times K_L \times K_L$ 的卷积核,在维度为 $I_C \times I_N \times I_N$ 的输入特征图上进行滑动窗口操作。在每个窗口内,卷积核与对应的输入特征图数据进行逐元素相乘后累加 (Multiply-Accumulate, MAC),最终生成输出特征图的一个像素点。由此可见,常规卷积的核心计算模式是在滑动窗口数据流下的高密度 MAC 运算。

1.2 深度可分离卷积

为构建轻量化网络模型,深度可分离卷积被提出以代替常规卷积,其计算过程如图 2 所示。它将标准卷积分解成两个串联的步骤:(1)深度卷积,每个输入通道独立地使用一个卷积核进行滤波。此过程与常规卷积类似,但卷积核与输入通道是一一对应的关系,其核心计算依然是在滑动窗口内的 MAC 操作。(2)点卷积,使用 1×1 的卷积核对深度卷积的输出进行通道融合。这在计算上等价于一次标准的 1×1 卷积,其本质也是高密度的 MAC 运算。

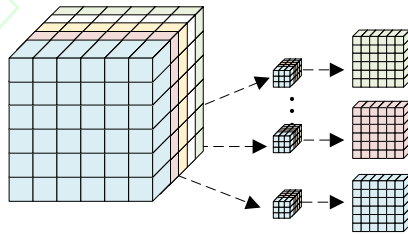


图 1 常规卷积示意图

Fig.1 Schematic diagram of conventional convolution

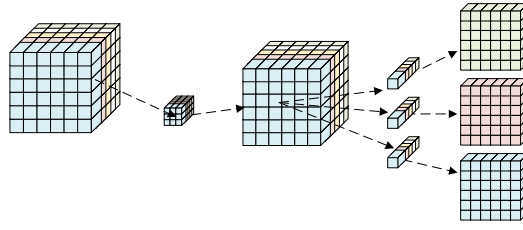


图2 深度可分离卷积示意图
Fig.2 Schematic diagram of depth-separable convolution

1.3 池化

池化层用于降低特征图的空间分辨率，从而减少后续网络层的参数量和计算量，能够与有效提取图像数据的局部特征。常见的池化操作有最大池化和平均池化，计算过程如图3所示。这些操作在一个局部窗口内进行，最大池化选取窗口内的最大值，平均池化计算窗口内数据的平均值，二者均可通过将一条长矢量划分为多段短矢量多次进行求最大值或求均值，来求得最后的长矢量的最大或平均池化结果。

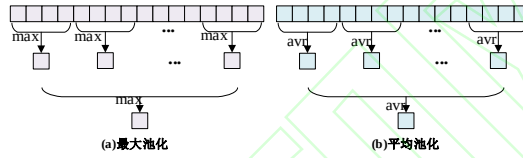


图3 池化示意图
Fig.3 Schematic diagram of pooling

1.4 全连接

全连接层通常位于CNN的末端，负责将高位特征映射到最终的输出空间。其计算式可表示为：

$$y = Wx + b \quad (1)$$

其中， x 是输入特征向量， W 是权重矩阵， b 是偏置向量。

该运算可以分解为一系列向量内积操作，每个内积运算均由多次乘法和以此累加构成。因此，全连接层的核心计算同样可以归结为大规模的MAC操作。

由以上描述可以看出，常规卷积、深度可分离卷积以及全连接核心运算的关键算子均是MAC计算，设计专用的矢量处理器架构以及专用指令集来对MAC计算进行并行化加速，是加速卷积计算的关键所在。

2 ASIP 架构设计

通过对上述CNN核心层算法分析可见，虽然各层功能不同，但是其底层计算存在明显共性。常规卷积和深度可分离卷积以及全连接层占据了CNN约95%的计算量，而这些计算密集型操作的核心均可归结为大规模的MAC运算。针对CNN加速设计的关键在于高效支持并行化MAC运算，设计专用的矢量MAC指令，结合能够支持高吞吐量数据流动的矢量寄存器和存储器访问机制。

为了减少在终端设备部署CNN时的存储、功耗和计算开销，通常会将CNN模型进行整数量化（如8位整数量化），从而减少内存占用加快计算速度（整数计算比浮点运算更快）并且降低功耗。

因此本文结合指令级并行和数据级并行技术，设计了一个基于VLIW+SIMD架构的七级流水线矢量处理器，处理器总体架构如图4所示。通过加长流水线级数，减小关键路径长度，提高处理器运行频率，从而加快处理器计算速度。综合考虑处理器的面积、能耗和计算性能，VLIW宽度设为6个指令槽，允许同一时钟周期内最多发射6条指令。其中Slot1和Slot2存放标量计算指令和控制指令，Slot3和Slot4专用于执行矢量计算指令，而Slot5和Slot6负责数据存取指令，通过将不同类型的指令放在不同指令槽中，可以避免执行时功能单元冲突，最大化利用指令并行度。处理器的SIMD宽度

设置为 256 比特，ASIP 支持的数据类型包括 32 位标量数据（存放于 R 寄存器）以及为适应 8 bit 量化模型的 32×8 位的矢量数据（存放于矢量寄存器 V）。但由于计算过程中的 MAC 操作会产生大于 8 位的中间计算结果，因此设计中还包含了一组 32×32 位的矢量寄存器 X 用于暂存临时数据。

ASIP 中集成了一个 DMA 模块，作为内存控制器，仲裁片上存储的访问，控制处理器运行状态以及处理片外 DRAM 存储器与片内 SRAM 的数据传输。片外接口根据 AXI 协议设计，提高传输效率，并且易于集成到主流 AMBA 架构的 SOC 结构。

ASIP 的程序代码存储在专用的 32 kB 单端口 SRAM 中，称为程序存储器（PM）。而输入特征图数据以及计算中间结果存储在 256 kB 的真双口 SRAM 中称为数据存储器（DM），CNN 的参数则单独存放在 32 kB 的伪双口 SRAM 中，称为参数存储器（CM）。SRAM 往往无法一次性容纳整个 CNN 层的特征图或权重，因此必须分块。

针对 DM，其 256 kB 的容量设计核心逻辑在于实现层间数据片内驻留，并最小化 DMA 调度频率。在双缓冲机制下，DM 被划分为两组 128 kB 的逻辑空间。

对于中小型特征图，128 kB 的单组深度足以容纳 MobileNet、ResNet 等主流模型中多数中间层的完整特征图。这意味着前一层计算输出可以直接存储在 DM 的一组 Buffer 中，并在下一层计算时直接作为输入读取，实现两组 Buffer 间的闭环迭代。这种设计使得处理器在处理连续多个网络层时，无需通过 DMA 将中间结果写回片外 DRAM，极大程度地消除了非必要的片外访存延迟与功耗。

对于超大特征图，即使面临无法完全驻留的层次，128 kB 的大容量也保证了更粗粒度的分块。较大的分块尺寸显著减少了 DMA 的启动总次数，规避了由于 AXI 总线频繁握手及外部存储器竞争导致的流水线停顿，从而在系统层面提升了任务执行的连续性。

针对 CM，单组逻辑容量设定为 16 kB，旨在利用带宽冗余实现访存延迟的完全隐藏。由于卷积核参数量通常远小于特征图，且具有极高的复用率，16 kB 的缓冲区足以支撑单次分块运算所需的权重。基于处理器的计算密度分析，DMA 搬移 16 kB 权重的时间远快于处理器消耗 128 kB 数据的时间。因此，即便 CM 容量较小，也能确保参数在计算开启前提前就绪。将 CM 维持在 16 kB 而非更小，是为了在节省硅面积的前提下，尽可能覆盖更多算子的参数需求，从而进一步减少与 DMA 的交互频率，降低系统总线的控制开销。

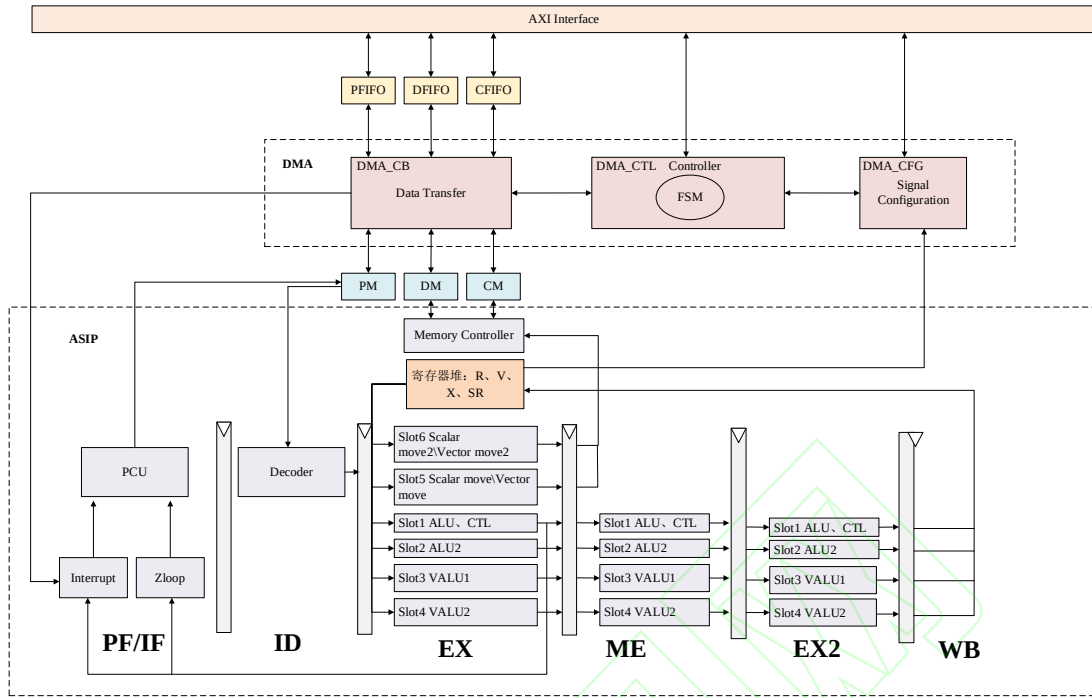


图4 处理器总体架构图
Fig.4 Overall architecture diagram of the processor

2.1 零开销循环结构

由于 CNN 的核心计算都涉及到多层循环的嵌套, 因此本文 ASIP 中, 设计了零开销循环模块, 通过硬件的直接参与来消除软件程序中由于循环嵌套引入的大量条件分支跳转指令导致的额外时钟周期。在本文处理器中, 设计了 Doloop 和 Zloop 两种零开销循环, 两者区别在于, 对于 Doloop, 循环起始地址是 do 指令的下一条指令地址, Doloop 共有两条指令, 其汇编代码格式为 *do s1, #imm_le* 和 *doi #imm_lc, imm_le*, 第一指令的循环次数来源于寄存器, 循环结束地址来源于立即数, 而第二指令的循环次数和循环结束地址均来源于立即数。Doloop 未被开启时, 循环标志寄存器 LF 的值设置为 3, 当在流水线 IF 阶段读取到 do 指令时, LF 被设置为 0 或 1, LS 保存循环起始地址, LE 保存循环结束地址, LC 保存循环次数。而 Zloop 没有专用循环启动指令, 它的循环起始地址可以在程序存储器的任意位置, 当循环开始地址, 循环结束地址以及循环次数被写入 ZS, ZE, ZC 后, 一旦 PCU 在流水线 PF 阶段发现当前程序地址是 Zloop 结束地址, 并且循环次数不为 0, 则跳回 Zloop 起始地址。此外, Doloop 最多可支持两层的零开销循环嵌套, Zloop 只支持一层的零开销循环, 因此, 两者结合使用最多可支持三层的零开销循环嵌套, 能够显著减少由于循环嵌套带来的时钟周期开销, 降低计算时延。

以一个三层循环嵌套为例分析零开销循环带来的性能提升, 嵌套层数分别是 M 、 N 、 K 。若不采用零开销循环, 每次循环末尾需要执行条件分支指令。由于每次执行分支跳转指令需要 3 个时钟周期, 因此总额外开销为:

$$T_{\text{traditional}} = 3 \times (M + M \times N + M \times N \times K) \quad (2)$$

在 K 值较大时, 由于分支指令被高频执行, 计算能效将显著下降。采用零开销循环方案时, 循环跳转由硬件控制器逻辑实现, 不占用执行周期。其唯一开销来源于循环启动前的寄存器配置指令, 其汇编指令示意图如图 5 所示。可以看出, 外层 Doloop 需要配置一次, 中层 Doloop 配置 M 次, 内层 Zloop 配置三个寄存器, 需要 3 个时钟周期, 因此零开销循环的总额外开销为:

$$T_{\text{ZOL}} = M + 4 \quad (3)$$

采用零开销循环架构可节省的时钟周期数为:

$$T_{ZOL} - T_{traditional} = 3 \times M \times N \times (1 + K) + \frac{2 \times M - 4}{2 \times M - 4} \quad (4)$$

可以看出, 该设计可以几乎完全消除分支跳转指令执行带来的额外时钟周期开销, 降低处理器计算时延。

为量化评估零开销循环机制对处理器执行效率的提升, 本文构建了涵盖三类不同嵌套深度的测试程序进行对比实验。为排除 VLIW 并行度的干扰并严格控制变量, 实验仅激活标量执行槽 Slot1 的计算指令。

测试结果如表 1 所示, 在单层循环场景下, 零开销循环通过硬件预取与自动跳转机制, 消除了传统循环中分支指令导致的流水线空泡, 实现了 5.89 倍的性能提升; 随着循环嵌套层数增加至两层与三层, 加速比分别进一步扩大至 7.94 倍与 11.96 倍。实验数据表明, 传统循环在处理嵌套结构时, 每一层循环的边界判定与地址回跳都会引入显著的流水线停顿, 而零开销循环机制能彻底消除这些非计算开销。由于嵌套结构对循环开销具有累积效应, 该机制在处理深层嵌套算法时展现出随层数加深而愈发显著的性能增益, 有力地支撑了其在 CNN 推理任务中的加速优势。

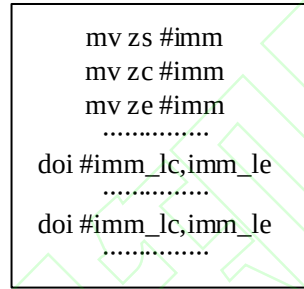


图 5 零开销循环汇编指令图

Fig.5 Zero-overhead loop assembly instruction diagram

表 1 零开销循环性能对比

Table 1 Comparison of Zero-Overhead Loop Performance

	计算时延 (Clock Cycles)		
	1 层	2 层	3 层
零开销循环	115	2131	34964
无零开销循环	678	16934	418342

2.2 矢量功能单元

矢量功能单元 (VALU) 位于 Slot3 和 Slot4 中, 用于实现数据级并行计算, 支持单时钟周期对矢量寄存器中的 32 路数据并行处理。针对资源受限的边缘端应用, 本文在并行度设计上进行了深入的设计空间探索与权衡分析: 虽然 16 路并行度具备更优的调度灵活性与更小的物理面积, 但其计算吞吐量难以满足实时推理的性能需求; 反之, 若将并行度盲目扩展至 64 路, 不仅其硬件资源开销将超出边缘端芯片的面积预算, 且在应用 DLLRP 算法加速卷积计算时, 若特征图宽度较小, 过宽的 SIMD 结构会导致矢量有效填充率下降, 并迫使处理器执行更多复杂的链接移位操作以重构目标操作数, 从而增加了额外的时钟开销。因此, 综合考虑面积约束、计算效率及算法执行效率, 本文最终将数据并行度确定为 32。并且设计时采用了数据旁路技术, 通过建立一条旁路路径, 将当前计算结果直接转发给后续指令, 如将矢量加法结果从 EX 直接转发到 ID, 而不需要等待结果写回寄存器中, 避免了数据冒险和流水线停顿, 提高了处理器的执行效率。指令的具体应用将在第 4 节介绍。

vshift、vlink、shuffle 指令是根据 DLLRP 算法设计的三条专用指令, 这些指令与 VALU 模块紧密映射, 以充分利用其并行计算能力。具体而言, vshift 指令在 VALU 的 Slot3 中通过专用移位逻辑单元执行, 该单元根据立即数参数选择移位模式, 如左移或右移; vlink 指令同样在 Slot3 中执行, 利用链接逻辑单元根据立即数实现数据链接重构; shuffle 指令则映射到 VALU 中的混洗单元, 该单元在 EX 阶段调用矢量重排逻辑, 支持 stride=2 的卷积运算, 通过 SIMD 并行处理实现高效数据洗牌。

`max_pool` 和 `avr_pool` 指令可以有效利用处理器的 SIMD 架构, 用于并行计算最大池化层或平均池化层, 这些指令在 VALU 的 Slot3 和 Slot4 中通过比较或求和逻辑单元执行, 在 EX 阶段并行处理 32 路数据。其余矢量专用指令还包括了矢量乘法 `vmul`、矢量加法 `vadd`、矢量激活 `vrelu`, 这些指令的调用逻辑均在 ID 阶段解析后, 在 EX 阶段调度相应的功能单元执行计算操作。

2.3 中断功能单元

中断功能单元 (Interrupt Functional Unit, IFU) 为处理器提供了响应外部设备请求的能力, 允许暂停当前执行流程以优先处理紧急请求。本文设计的中断控制模块支持最多 8 个中断服务。该功能单元在 CNN 计算场景中承担关键的同步功能。当处理器完成一组数据的计算后, 为等待 DMA 将下一组参数从外部存储器传输至 CM, 处理器可进入一种低功耗休眠状态, 并将下一条指令的地址保存至中断链接寄存器 ILR。当 DMA 完成数据传输后, DMA 控制器会发出一个特定的中断信号恢复处理器的计算。为了使处理器能正确响应中断, 程序初始化阶段需要进行配置。通过执行 `ei` 指令将中断使能寄存器的值置 1, 全局性地允许中断处理。然后, 需要配置 8 位的中断掩码寄存器 IM, 通过 `mv im, 0x02` 指令将 IM 第二位位置 1, 其余位置为 0。代表处理器仅会中断服务向量表中的二号中断请求, 而忽略其它中断请求, 屏蔽其它潜在中断源, 实现精准同步控制。中断服务向量表的二号中断请求有效时, IFU 会控制处理器执行 2 号中断服务程序 `rti` (Return From Interrupt) 指令, 该指令会根据 ILR 中保存的地址, 跳转回休眠前的下一条指令处开始执行, 继续执行下一组数据的计算。中断请求输入电路结构如图 6 所示。

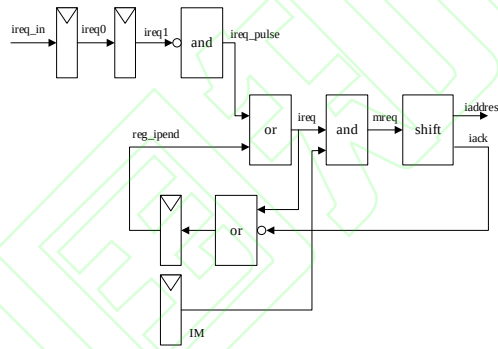


图 6 中断请求输入电路图
Fig.6 Input of interrupt request circuit diagram

2.4 DMA 功能单元

本文设计的 DMA 模块主要包括 DMA_CFG、DMA_CTL、DMA_CB 三个子模块。其中 DMA_CFG 用于管理配置寄存器, 接收来自 AXI 接口的配置信息如开始命令信号、DMA 读写数据地址以及读写数据长度。DMA_CTL 模块则用来控制当前 ASIP 工作状态, 当 DMA_CTL 接收到来自 DMA_CFG 的配置信息后, 判断处理器是否需要工作状态转移, 并将当前工作状态的控制信息传输给 DMA_CB 模块。DMA_CB 根据这些控制信息来进行 FIFO 和 SRAM 间的数据读取操作。由于 CNN 各网络层中不同输出通道的计算是相互独立的, 因此为了减少片上存储资源的开销, DMA 采用乒乓 buffer 的传输机制向 CM 中传输数据。当处理器在进行第一组数据计算的同时, DMA 继续向 CM 第二块存储中传输下一组计算所需的数据。这样的计算传输机制不仅减少了存储资源消耗, 同时减少甚至消除了处理器等待参数数据传输的时间, 提高了存储利用率和计算效率。

3 基于 ASIP 矢量架构的 CNN 加速算法实现

在第二章中, 本文构建了一个基于 VLIW+SIMD 架构的矢量处理器, 其核心计算能力建立在 256 位宽的矢量数据通路上。该架构虽然通过 32 路并行 MAC 运算提供了强大的峰值算力, 但也对数据的组织方式提出了严苛要求。

为减少复杂的访存逻辑和资源开销, 本文设计的处理器采用基于标量地址寻址的 SIMD 数据通路, 其 `vload` 指令支持单周期加载 32 个连续存储的数据。然而, 由于卷积运算固有的滑动窗口机制导致频繁的非对齐和跨行数据访问, 与 SIMD 架构倾向处理连续内存数据的特性不匹配。因此, 在采用常规卷积计算流程的情况下, SIMD 架构无法直接有效地对其加速计算。

此外, 在常规卷积和深度卷积实现中, 为了维持输入与输出特征图空间维度一致, 需要在输入特征图周围进行零填充 (Zero-Padding), 如图 7 所示。但在硬件实现中, 当特征图数据驻留于片上高速缓存时, 执行显示的零填充不仅会引入额外的写操作延迟和硬件资源消耗, 同时还增加了内存管理和地址生成逻辑的复杂性。

为解决上述问题, 本文提出一种数据层链接重构并行计算方法 (Data Layer Link Reconstruction Parallel Computing Method, DLLRP), 充分利用了 ASIP 的并行计算结构, 通过优化数据流来免去卷积运算中的非连续访存过程, 并消除物理零填充引入的开销。DLLRP 的核心思想是, 在寄存器级对加载的连续数据进行重构, 以模拟滑动窗口和 Zero-Padding 的效果, 生成卷积核不同位置所需的矢量操作数, 从而替代非连续访存和显式零填充操作, 避免了物理零填充设计额外的控制逻辑与存储单元, 从而在减少访存次数的同时降低了硬件开销。优化后的数据流使得处理器能对连续内存地址的数据高效执行并行计算, 进而提升计算效率, 显著降低了卷积运算的时延。

3.1 数据层链接重构并行计算方法

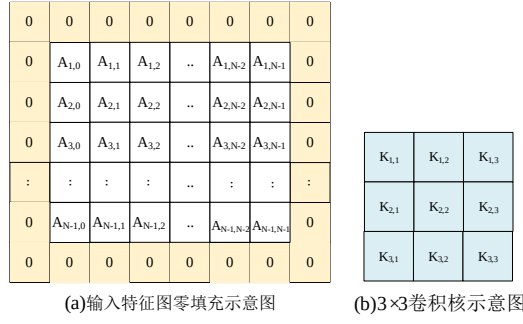
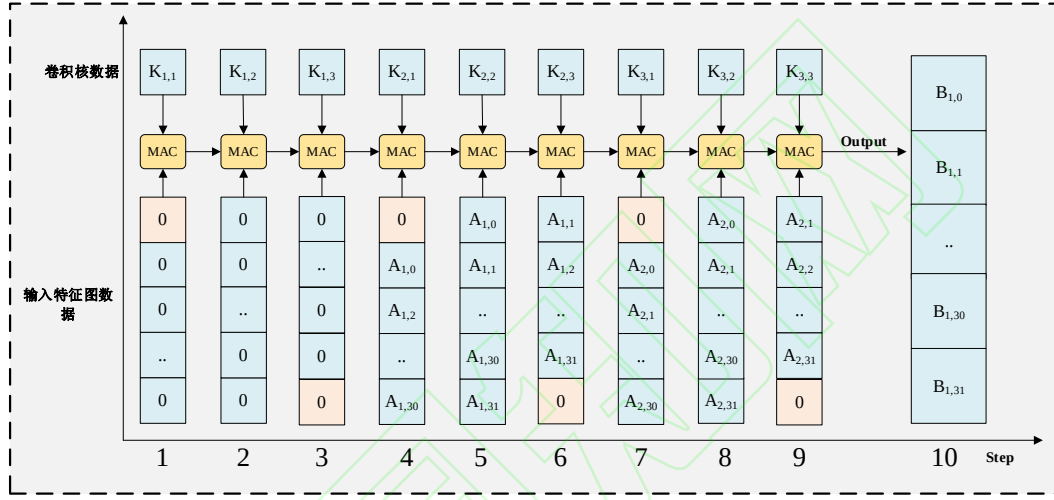
以一个深度卷积计算为例, 假设输入特征图尺寸为 32×32 , 卷积核大小为 3×3 , DLLRP 计算流程如图 8 所示。可见, DLLRP 使用 `vload` 获取一行连续数据后, 后续通过寄存器内矢量移位和零填充操作, 即可生成卷积核计算所需的矢量数据片段。这些矢量数据与对应卷积核权重相乘, 再将所有卷积核权重的计算结果累加, 就能得到输出特征图的一行数据。

具体而言, 输出特征图第一行的计算涉及到输入特征图的前两行以及概念上存在于输入特征图上方的一行零填充数据。当计算卷积核第一行 ($K_{1,1}, K_{1,2}, K_{1,3}$) 于输入数据的乘积时, 其对应的输入数据实际上是零填充区域, 因此这些计算结果为零, 相关的计算可被优化省略。对于卷积核第二行 ($K_{2,1}, K_{2,2}, K_{2,3}$) 计算所需数据, 可以通过加载输入特征图第一行的矢量数据来构造生成。 $K_{2,1}$ 乘上的数据, 对应于输入特征图第一行数据向右逻辑移位一位, 并在左侧补零。 $K_{2,2}$ 乘上的数据正好是输入特征图第一行数据本身, 而 $K_{2,3}$ 乘上的数据对应于输入特征图第一行数据向左逻辑移位一位, 并在右侧补零。卷积核第三行 ($K_{3,1}, K_{3,2}, K_{3,3}$) 所需数据的生成方式与第二行类似, 只是操作数变为了输入特征图的第二行数据。

为支持这种数据重构, 设计了专用矢量指令 `vshift d1, s1, #imm`, 其功能是根据立即数指定的模式, 对源寄存器 `s1` 中的向量进行移位和选择性零填充, 结果写入目标寄存器 `d1`。这种方式有效提升了加载数据的复用性和计算并行度, 显著加速了卷积运算的执行速度。

在访存开销方面, DLLRP 算法展现出明显优势。由图 8 分析可知, 生成一行输出特征图仅需 9 次标量访存指令 (`load`) 读取权重, 以及 3 次矢量访存指令 (`vload`) 读取支撑计算的三行输入特征数据。相比之下, 传统的 `im2col` 计算方式为生成相同的输出行, 首先需要 1 次 `vload` 加载卷积核权重, 随后必须通过滑动窗口逐点提取输入特征并重排为列向量, 这一过程通常涉及 32 次矢量访存操作, 总访存次数高达 33 次。定量分析表明, DLLRP 的访存需求仅为传统 `im2col` 算法的 36.36%, 大幅减少了访存次数, 缓解带宽压力。

此外, DLLRP 算法完全基于连续地址访存, 而 `im2col` 在处理非对齐边界或跨行数据提取时, 往往需要依赖类似于 RISC-V 扩展指令集中的 `gather/scatter` 指令。这类指令在硬件实现上涉及复杂的地址计算和非对齐内存访问冲突 (Bank Conflict), 通常难以在单一时钟周期完成完整的矢量提取。相比之下, DLLRP 算法的连续地址访存, 配合专用指令 `vshift`, 不仅简化了硬件控制逻辑, 更通过减少访存延迟和指令周期数, 实现整体计算效率的提升。

图 7 输入特征图零填充和 3×3 卷积核示意图Fig.7 Schematic diagram of zero-padding for input feature maps and 3×3 convolution kernelsFig.8 32×32 Depthwise convolution DLLRP computation flowchart

同时, `vshift` 指令的移位补零规则具有良好的规律性和可扩展性, 计算式如式 5 所示, 其中 $s1 = A_{0 \sim 31}$ 。由于本文设计 ASIP 的 SIMD 宽度固定为 32, 当输入特征图尺寸不等于 32×32 时, 计算单行输出特征图数据需要多个 SIMD 矢量数据参与。DLLRP 方法针对输入数据的移位和零填充逻辑在原理上与 32×32 输入的情况保持一致。然而由于卷积核的滑动窗口会跨越两个相邻的 SIMD 数据块边界, 直接在单个 `vload` 的矢量上应用 `vshift` 不能获取完整的卷积操作数。因此, 本文设计了矢量链接指令 `vlink d1, s1, s2, #imm`。该指令的核心功能是根据立即数 `imm` 指定的模式, 选择性拼接来自两个源寄存器 `s1` 和 `s2` 的矢量数据, 并将跨越这两个数据块边界的、卷积计算所需的局部数据提取重构到一个目标矢量 `d1` 中, 以生成可以直接用于矢量 MAC 计算的操作数, `vlink` 指令计算式如式 6 所示, 其中 $s1 = A_{0 \sim 31}$, $s2 = A_{32 \sim 63}$ 。通过 `vlink` 指令, ASIP 可以并行地计算比 SIMD 更低维度或更高维度的输入特征图, 保证了 DLLRP 计算方法的可扩展性。

$$vshift(s1, i) = \begin{cases} [0, A_{0 \sim 30}], & i = 0 \\ [A_{1 \sim 31}, 0], & i = 1 \\ [0, A_{0 \sim 14}, 0, A_{16 \sim 30}], & i = 2 \quad (5) \\ [A_{1 \sim 15}, 0, A_{17 \sim 31}, 0], & i = 3 \\ [0, A_{0 \sim 6}, 0, A_{8 \sim 14}, 0, A_{16 \sim 22}, 0, A_{24 \sim 30}], & i = 4 \\ [A_{1 \sim 7}, 0, A_{9 \sim 15}, 0, A_{17 \sim 23}, 0, A_{25 \sim 31}, 0], & i = 5 \end{cases}$$

$$vlink(s1, s2, i) = \begin{cases} A_{31 \sim 62}, & i = 0 \\ A_{1 \sim 32}, & i = 1 \\ A_{16 \sim 47}, & i = 2 \quad (6) \\ [A_{0 \sim 7}, A_{32 \sim 55}], & i = 3 \\ [A_{0 \sim 23}, A_{32 \sim 39}], & i = 4 \end{cases}$$

当卷积的步长 (stride) 等于 2 时, 卷积核在输入特征图上滑动时访问的操作数不再是空间上紧邻的, 以 $stride=2$ 在 64×64 输入特征图上的卷积为例, 卷积核每次移动会跳过一个像素, 导致与其相乘的输入数据在内存中是非连续但具有规则间隔的。

为高效处理这种跨步卷积模式, DLLRP 方法引入了矢量混洗指令 $shuffle\ d1, d2, s1, s2$, 该指令的计算式如式 7 所示。通过这种奇偶混洗模式, 将原本在内存中间隔的操作数收集并排列到目的矢量寄存器 $d1$ 和 $d2$ 中。利用 $d1$ 和 $d2$ 中的矢量数据, 结合 $vshift(s1, 1)$ 和 MAC 指令, 即可在 SIMD 架构上高效执行跨步卷积。

$$shuffle(s1, s2) = [A_{2i}, A_{2i-1}], i = 1 \sim 32 \quad (7)$$

3.2 点卷积并行计算方法

由于点卷积的卷积核尺寸为 $N \times 1 \times 1$ (N 为卷积核通道数量), 点卷积计算不存在非连续访存过程, 在本文设计的 ASIP 中, 点卷积的数据流能高效映射至 SIMD 执行单元。以输入特征图尺寸为 32×32 , 输入通道数为 8 的情况为例, 其计算流程如图 9 所示。A-H 代表输入特征图的 8 个通道, $K_1 - K_8$ 代表第一个输出特征图对应的第一组点卷积核, 可以看出, 点卷积的并行计算过程仅通过矢量存取和矢量乘累加指令即可高效完成。并且由于点卷积没有非连续访存过程, 因此对于更高维和更低维的输入特征图, 本文设计的 ASIP 均能直接实现并行计算。

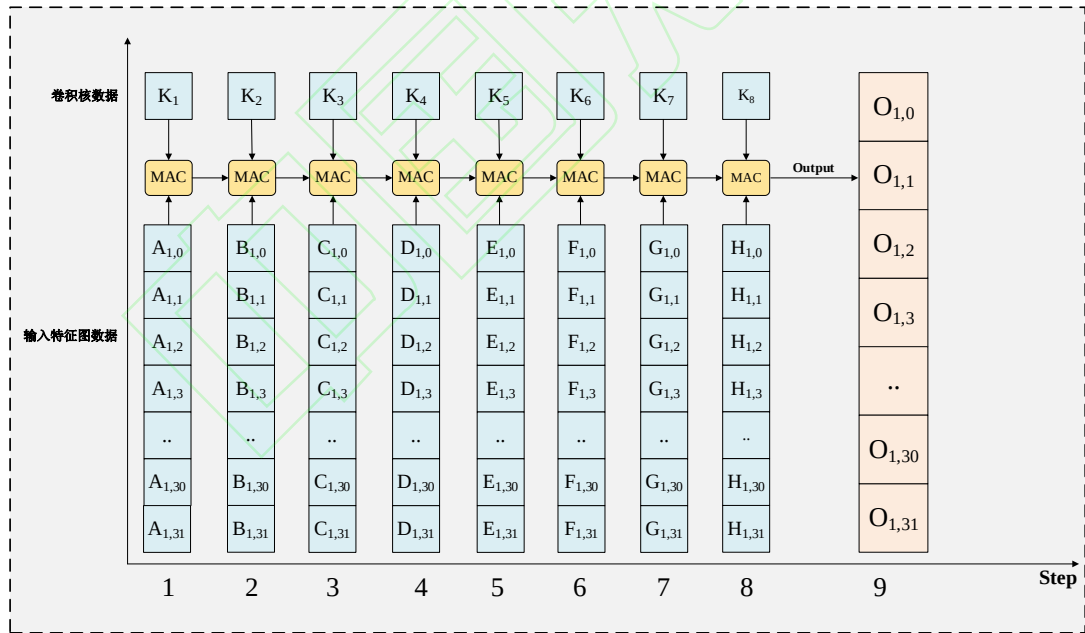


图 9 32×32 点卷积示意图

Fig.9 Schematic diagram of 32×32 point convolution

4 模型部署及 ASIP 性能评估

本文采用 Synopsys 公司的 ASIP Designer T-2022.06 工具套件进行 ASIP 的开发和实现。针对基于 ImageNet 数据集训练且分辨率超参数 $\rho=0.571$ 的 MobileNetV1 量化模型, 完成了在 ASIP 上的部署与功能验证。通过 ASIP Designer 工具的 Debug 套件, 编写 MobileNetV1 量化模型的 C 语言实现代码,

进行软件功能仿真后, Debug 套件内置的编译器可将 C 语言程序翻译为 ASIP 指令集对应的汇编程序, 再通过链接器生成可供 ASIP 读取的二进制程序文件, 存放在 PM 中, 用于后续 ASIP 的 RTL 级和门级网表级仿真时读取执行。

在硬件实现方面, 基于 TSMC 28nm 工艺库对 ASIP 进行逻辑综合和布局布线, 综合结果如表 2 所示。关键路径延时为 0.97 ns, 最高时钟频率能够达到 1.03 GHz, ASIP 总面积(包括 SRAM)为 1.229 mm², NAND-2 等效门数为 2.438 MGE。在 1 GHz 的运行频率下, ASIP 峰值性能可达 130 GOPs, 完成 MobileNetV1 模型前向推理任务的时间为 12.8 ms, 实现了 85.93 GOPs 的计算性能。

表 2 ASIP 实现参数
Table 2 ASIP Implementation Parameters

工艺	TSMC 28nm
电压[V]	1.0
面积[mm ²]	1.229
等效门数[MGE]	2.438
SRAM[kB]	320
时钟频率[GHz]	1.0
峰值性能[GOPs]	130

4.1 ASIP 加速性能评估与对比

鉴于端侧设备对硬件面积和功耗有着严格要求, 加速器的面积效率和能量效率是衡量其性能的关键指标。在表 3 中, 将所提出的 ASIP 设计与其它文献中应用于边缘设备的 CNN 加速器在面积效率方面进行了对比分析。考虑到各研究工作采用的工艺制程不同, 为确保比较的基准一致性与结果的客观性, 本文统一采用 NAND-2 等效门数作为衡量面积的归一化标准。在 1000 MHz 的工作时钟频率下, 本文设计的 ASIP 实现了 53.32 GOPs/MGE 的峰值面积效率。与相关研究相比, 该数值分别为文献[6]、[8]、[9]、[7]及[11]面积效率值的 1.47 倍、1.28、5.80 倍、1.55 倍和 1.29 倍。表明本设计在硬件资源利用率方面具有一定的优势。

在能效评估方面, 本文使用 PTPX 工具对所设计 ASIP 在推理 MobileNetV1 网络模型时的平均功耗进行了测量。在设定工作电压为 1V、ASIP 时钟频率为 1000 MHz 的条件下, 抓取了 150 ns 的 VCD 波形用作功耗分析。MobileNetV1 网络的平均计算功耗为 258.8 mW, 由此可推算出 ASIP 在推理 MobileNetV1 网络时的能量效率为 332 GOPs/W。表 4 进一步将本 ASIP 设计与其它已发表的 CNN 硬件加速器方案进行了能效对比。文献[12]采用多电压域设计策略, 通过为各个子阵列动态调整供电电压来降低整体功耗, 然后其在 1.2V 工作电压下的能效为 45.8 GOPs/W, 本设计的能效是其 7.25 倍。另一项代表性研究 Eyeriss V2^[8]通过构建稀疏数据计算的处理单元, 来减少运算量和数据访问开销, 从而提升 CNN 计算速度。但 MobileNetV1 作为一种轻量化网络, 其固有层结构较为紧凑, 数据稀疏度不高, 且数据复用机会相对有限, 在一定程度上限制了 Eyeriss V2 计算单元的利用率。因此, 在推理 MobileNetV1 网络时, 本设计能效达到 Eyeriss V2 的 1.71 倍。文献[9]提出的加速器架构不仅支持常规卷积, 也高效处理深度可分离卷积, 且通过卷积核切分与 PE 单元的协同设计, 使得在不同卷积核尺寸下都能接近 100% 的 PE 硬件利用率, 其在 500 MHz 时钟频率下, 能达到 213.3 GOPs/W 的能效, 本文方案是其 1.56 倍。ConvAix^[11]作为另一款 ASIP, 采用了 8-VLIW 与可变精度 SIMD 的并行计算架构, 支持 4 位、8 位及 16 位等多种精度的矢量运算。尽管 ConvAix 在处理 AlexNet、VGG16 及 ResNet-16 等传统网络时展现出优良的计算效率, 但由于 MobileNetV1 网络广泛采用的深度可分离卷积的并行计算模式, 与其针对标准卷积优化的数据复用模式存在不匹配, 进而导致其 VALU 整体利用率有所下降。因此, ConvAix 在 513 MHz 条件下推理 MobileNetV1 网络的能量效率为 256.3 GOPs/W, 显著低于其处理 VGG16 网络时的 675.6 GOPs/W。相较之下, 得益于本文提出的 DLLRP 计算方法对于深度卷积运算的显著加速效应, 本文 ASIP 设计的能效是 ConvAix 的 1.30 倍。文献[5]基于片上网络的方案优化数据流, 面向轻量化神经网络进行加速, 在 200 MHz 工作频率下能达到 126.9 GOPs/W 的能效, 仅为本文设计方案的 38.2%。

此外, 为进一步验证架构的综合推理效能, 本文将所设计的 ASIP 与商用高性能 RISC-V 矢量处

理器 Andes NX27V2^[22]进行了对比。实验结果显示,在同为 1 GHz 的工作频率下,Andes NX27V2 执行 MobileNetV1 模型推理的延迟为 42.2 ms,而本文提出的 ASIP 架构仅需 12.8 ms。相较于通用 RISC-V 矢量扩展架构,本文设计的专用处理器在端侧典型 CNN 任务中展现出更优的计算效率与吞吐能力。

并且,由于常规的卷积算法在计算过程中,矩阵乘法是 CNN 计算性能的关键瓶颈,许多现有研究常用矩阵乘法运算的加速性能作为评估深度学习加速器对 CNN 整体加速能力的指标。本节通过矩阵乘法基准,将本文提出的 ASIP 与两款基于 RISC-V 指令集架构的深度学习加速器进行对比,如表 5 所示。结果显示,本文方案的性能分别为文献[22]的 1.18 倍和文献[23]的 2.10 倍。这一比较进一步验证了本文设计的 ASIP 在 CNN 加速领域具备优异的性能表现。

本研究通过对所设计的 ASIP 与现有文献中的多种 CNN 加速器在面积效率和能量效率两个维度进行定量对比分析,结果表明本文方案展现出良好的综合性能优势,能够在边缘计算设备面临硬件资源受限条件下保持高效的运算性能。此外,通过将 ASIP 的矩阵运算性能与其它高性能加速器进行对比,进一步验证了本文设计在计算效率方面的高效性。

表 3 不同加速器性能对比
Table 3 Comparison of Performance of Different Accelerators

	JSSC 2017 Eyeriss ^[6]	JETCAS 2019 Eyeriss V2 ^[8]	ICCSC 2021 ^[9]	TCSVT 2017 Origami ^[7]	OJCAS 2021 ConvAix ^[11]	Ours
工艺制程 (nm)	65	65	40	65	28	28
测量方案	Silicon	Silicon	Silicon	Silicon	PnR	PnR
On-Chip SRAM (kB)	183.3	246	-	344	160	320
电压 (V)	1.0	-	0.9	0.8	1.0	1.0
时钟频率 [MHz]	200	200	500	189	513	1000
峰值性能 (GOP/s)	67.2	153.6	32	74	262.6	130
面积 (mm ²)	12.3	-	-	3.09	3.53	1.229
面积 (MGE)	1.852	3.97	3.48	2.157	6.38	2.438
面积效率 (GOPs/mm ²)	5.46	-	-	23.82	74.39	105.78
面积效率 (GOPs/MGE)	36.29	41.51	9.19	34.31	41.16	53.32

表 4 不同加速器的能效对比
Table 4 Energy Efficiency Comparison of Different Accelerators

	AICAS 2021 ^[12]	JETCAS 2019 Eyeriss V2 ^[8]	ICCSC 2021 ^[9]	OJCAS 2021 ConvAix ^[11]	ASICON 2023 ^[5]	Ours
工艺制程 (nm)	65	65	40	28	40	28
测量方案	PnR	Silicon	Silicon	PnR	Silicon	PnR
On-Chip SRAM(kB)	216	246	-	160	68	320
量化精度 (bit)	8	8	8	8	8	8
电压 (V)	0.45-1.2	-	0.9	1.0	1.1	1.0
时钟频率 (MHz)	-	200	500	513	200	1000
测试模型	MobileNetV1	MobileNetV1	MobileNetV1	MobileNetV1	MobileNetV1	MobileNetV1
功耗 (mW)	-	-	150	313.1	301.7	258.8
能效 (GOPs/W)	45.8	193.7	213.3	256.3	126.9	332.0

表 5 矩阵乘法加速性能对比
Table 5 Performance Comparison of Matrix Multiplication Acceleration

矩阵大小	计算时延 (Clock Cycles)			加速比例
	TCAS I 2025 ^[23] (T ₁)	CF '25 2025 ^[24] (T ₂)	Ours (T ₃)	
32×32	1566	-	1324	1.18 (T ₁ /T ₃)
64×64	-	17676	8412	2.10 (T ₂ /T ₃)

5 总结

为应对 CNN 在资源受限的边缘设备上部署时,面临的计算性能、存储开销与模型部署灵活性的挑战,本文设计并实现了一款应用特定指令集处理器 (ASIP)。该 ASIP 通过深度优化的并行计算架构、创新的 DLLRP 算法以及专用矢量指令集的协同设计,有效提升了计算效率并增强了对不同网络模型的适应能力,可高效加速包括常规卷积、深度可分离卷积、池化及全连接层在内的多种 CNN 层,各层均支持并行计算。在 1000 MHz 工作频率下,ASIP 达到了 53.32 GOPs/MGE 的峰值面积效

率,并在 MobileNetV1 网络推理任务中,达到了 332 GOPs/W 的能效。这些数据反映了 ASIP 在硬件资源利用和能耗控制方面的高效性,并且凭借其可编程特性,为不同应用场景下各类神经网络模型的快速迭代和灵活部署提供了有效支持。

后续工作将进一步验证并扩展 ASIP 对多样化边缘侧负载的支持能力。基于现有的可编程指令集架构,通过高效映射 MobileNetV2 与 MobileNetV3 中的倒残差结构及线性瓶颈层,并优化对特定激活函数的支持,旨在实现多种先进轻量化神经网络在本文架构上的快速迁移与高效运行。

因此本文提出实现的 ASIP 设计在实现高面效和高能效的同时,兼顾了应用部署的灵活性与可扩展性,从而在资源受限的端侧智能计算领域展现出重要的实际应用价值与良好发展潜力。

参考文献

- [1] HONG H,CHOI D,KIM N, et al.Mobile-X: Dedicated FPGA implementation of the mobilenet accelerator optimizing depthwise separable convolution[J].IEEE Transactions on Circuits and Systems II: Express Briefs, 2024, 71(11): 4668-4672.
- [2] LEE S L,KIM H.GaussianMask: Uncertainty-aware instance segmentation based on Gaussian modeling[C]//2022 International Conference on Pattern Recognition. IEEE, 2022:3851-3857.
- [3] KUO J T,WU C B,LUO W X.A flexible and efficient hardware accelerator architecture design for multiple CNN[C]//2023 IEEE International Conference on Consumer Electronics-Asia. IEEE, 2023:1-4.
- [4] SONG Y S,LEE K Y.A Design of lightweight convolutional neural network accelerator for IoT devices[C]//2023 Fourteenth International Conference on Ubiquitous and Future Networks. IEEE, 2023:474-477.
- [5] GAO J,SHAO Q,DENG F, et al.An NoC-based CNN accelerator for edge computing[C]//2023 IEEE 15th International Conference on ASIC. IEEE, 2023:1-4.
- [6] CHEN Y H,KRISHNA T,EMER J S,et al.Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks[J].IEEE Journal of Solid-State Circuits, 2017, 52(1):127-138.
- [7] CAVIGELLI L,BENINI L.Origami:A 803-GOp/s/W convolutional network accelerator[J]. IEEE Transactions on Circuits and Systems for Video Technology, 2017, 27(11):2461-2475.
- [8] CHEN Y H,YANG T J,EMER J,et al.Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices[J].IEEE Journal on Emerging and Selected Topics in Circuits and Systems, 2019, 9(2): 292-308.
- [9] KUO K C,HOU H C.Deep neural network accelerator design for the multi-task, high performance, and energy-efficient edge computing[C]//2021 4th International Conference on Circuits, Systems and Simulation. IEEE, 2021:62-66.
- [10] HUANG B M,HUAN Y X,CHU H M,et al.IECA: An In-Execution Configuration CNN Accelerator With 30.55 GOPS/mm² Area Efficiency[J].IEEE Transactions on Circuits and Systems I: Regular Papers, 2021, 68(11):4672-4685.
- [11] BYTYN A,LEUPERS R,ASCHEID G.ConvAix: An Application-Specific Instruction-Set Processor for the Efficient Acceleration of CNNs[J].IEEE Open Journal of Circuits and Systems, 2021, 2:3-15.
- [12] HOSSAIN M S,SAVIDIS I.Energy efficient computing with heterogeneous DNN accelerators[C]//2021 IEEE 3rd International Conference on Artificial Intelligence Circuits and Systems. IEEE, 2021:1-4.
- [13] Kovačević N,Mišeljić D,Stojković A.RISC-V vector processor for acceleration of machine learning algorithms[C]//2022 Telecommunications Forum. IEEE, 2022:1-4.
- [14] WANG Z,DU H C,ZHOU J R,et al.GRS: A general RISC-V SIMD vector acceleration processor for artificial intelligence applications[C]//2024 IEEE Asia Pacific Conference on Circuits and Systems. IEEE, 2024:702-706.
- [15] HAN S,LIU X Y,MAO H Z,et al.EIE: Efficient inference engine on compressed deep neural network[C]//2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture. IEEE,2016:243-254.
- [16] MOONS B,VERHELST M.An energy-efficient precision-scalable convNet processor in 40-nm CMOS[J].IEEE Journal of Solid-State Circuits, 2017, 52(4):903-914.
- [17] BYTYN A,LEUPERS R,ASCHEID G.An application-specific VLIW processor with vector instruction set for CNN acceleration[C]//2019 IEEE International Symposium on Circuits and Systems. IEEE, 2019:1-5.
- [18] SONG Z F,ABDELAZIM A E,SABET P B,et al.A power-efficient Attention-Infused CNN hardware accelerator for RF spectrum monitoring[C]//2025 IEEE International Symposium on Circuits and Systems. IEEE, 2025:1-5.
- [19] Chen D F,LIU X H,SANG Y J,et al.A FPGA-Based accelerator design for speaker verification using 1D-CNN[C]//2024 International Conference on Electronic Information Engineering and Computer. IEEE, 2024:80-84.
- [20] YIN Z H,LIU B Q,WANG J P,et al.A low-hardware-overhead, high-energy-efficiency, and end-to-end CNN-based feature extraction accelerator for mobile visual SLAM[C]//2024 IEEE Asia Pacific Conference on Circuits and Systems. IEEE, 2024:115-119.
- [21] LI H R,GONG L,WANG C.A flexible dataflow CNN accelerator on FPGA[C]//2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing Workshops. IEEE, 2023:302-304.
- [22] WANG S T C.The Future of Edge Is About RISC-V and AI [EB/OL]. (2023-02-22) [2026-1-22]. https://www.andestech.com/wp-content/uploads/The-Future-of-Edge-is-About-RISC-V-and-AI_Simon-Wang.pdf.
- [23] M. Guo,WU Q,WANG C N,et al.A RISC-V Domain-Specific processor for deep Learning-Based channel estimation[J].IEEE Transactions on Circuits and Systems I: Regular Papers, 2025, 72(5):2380-2393.
- [24] Cammarata D,Perotti M,Bertuletti M,et al.Quadrilatero: A RISC-V programmable matrix coprocessor for low-power edge applications[C]//Proceedings of the 22nd ACM International Conference on Computing Frontiers: Workshops and Special Sessions. 2025: 66-69.